Agent Postmortem Generator

Part 10 - Testing Strategy & Test Infrastructure

Design document for the module that makes the rest of the system provable, reproducible, and hard to accidentally break.

Project name	Agent Postmortem Generator
Part number	10
Module name	Testing Strategy & Test Infrastructure
One-line purpose	Define the test layers, fixture system, and quality gates that keep the project deterministic, maintainable, and safe to change.

This document is intentionally module-scoped. It defines how testing is organized, not what product features exist.

2. Executive Overview

The Testing Strategy & Test Infrastructure module defines how every other module in the project is verified. It provides the test conventions, fixture formats, deterministic comparison rules, and shared utilities that let the team prove the behavior of parsing, normalization, analysis, postmortem generation, export, and the CLI without depending on a platform team.

This module exists because the project's value depends on trust. If the analysis engine, canonical trace model, or rendering pipeline changes silently, the tool stops being credible. The testing layer therefore acts as a quality contract: it catches regressions, preserves deterministic behavior, and keeps the MVP small enough for a few engineers to maintain.

In the system architecture, this module does not sit on the runtime path. It sits beside the codebase and governs the runtime path through tests, fixtures, golden outputs, and CI gates. In practice it underpins the entire repository.

3. Problem Statement

The project's core logic is highly structured but also easy to break in subtle ways. A small change to normalization can alter the canonical trace model. A small change to analysis can shift first-fault attribution. A small change to rendering can produce a report that still looks acceptable while silently losing evidence. These are not hypothetical problems; they are the normal failure mode of structured developer tools.

The hard part is not writing tests. The hard part is designing tests that are deterministic, stable across environments, and expressive enough to catch regressions without becoming noisy or impossible to maintain. The module must solve fixture drift, snapshot churn, path-specific failures, and inconsistent assertions across modules.

The MVP needs this module because the project is only useful if the output can be trusted. Without an opinionated testing foundation, the repository will accumulate ad hoc tests, weak assertions, and flaky CI. That would destroy the main product promise: a local-first tool engineers can rely on.

4. Goals

- Make the expected behavior of each module explicit, reproducible, and easy to verify.
- Provide a single shared testing vocabulary for units, integration flows, golden artifacts, and regression cases.
- Keep tests deterministic across developer laptops and CI environments.
- Make fixtures easy to create, review, update, and reason about.
- Catch behavior changes in the canonical trace model, analysis engine, and output generators before they reach users.

  Success means a new engineer can clone the repository, run the test suite locally, understand the failures, and fix them without guessing which layer is responsible.

5. Non-Goals

- This module does not define product features, parsing rules, or analysis logic.
- It does not introduce remote test services, cloud runners, or distributed test orchestration in the MVP.
- It does not build a generic test framework for arbitrary external projects.
- It does not add probabilistic test oracles or LLM-based verification.
- It does not own release engineering beyond the test-related gates needed for local development and CI.

  Later versions may add property-based fuzzing, broader corpus management, browser-based UI automation, and performance benchmarking at scale. Those are valid future concerns, but they are not required to establish the first reliable release.

6. Responsibilities

Owns	Does not own
Test conventions and repository-wide quality gates	Runtime parsing, normalization, analysis, or rendering logic
Shared fixtures, golden outputs, and deterministic comparison rules	Product-specific business rules
Unit/integration test boundaries and selection strategy	Deployment or release infrastructure beyond test execution
Test data hygiene, fixture versioning, and regression prevention	Framework adapters or user-facing UI behavior
Local and CI execution patterns for the test suite	Remote execution infrastructure in MVP

The module owns the system of verification, not the implementation being verified. Its job is to make boundaries observable and failures actionable.

7. Inputs

The module receives four kinds of inputs:

- Source code changes across the repository.
- Fixture files representing canonical traces, parser inputs, normalized traces, analysis outputs, and rendered artifacts.
- Developer and CI execution commands that select subsets of tests.
- Golden expectations such as JSON outputs, markdown reports, graphs, and exported files.

Validation rules

- Fixtures must be deterministic and versioned; no hidden dependence on current time, random seeds, network resources, or machine-local state.
- Golden artifacts must have a single canonical representation for comparisons.
- Test names must indicate the layer under test and the behavior being asserted.
- Shared fixtures must remain minimal and intentional; a large fixture is a maintenance cost, not a virtue.

Assumptions

- The codebase is a Python monorepo.
- The canonical trace model is stable enough to serve as the main test boundary.
- Most tests can run locally without network access.
- The project will use deterministic file paths and fixture roots.

Malformed input handling

Bad test inputs should fail loudly and close to the source. A malformed fixture, missing golden file, or invalid comparison target should be reported as a test setup error, not as a vague assertion failure deep inside helper code.

8. Outputs

- Pass/fail test results with clear failure localization.
- Human-readable failure messages that point to the broken boundary.
- Golden regression artifacts for normalized traces, analysis outputs, reports, graphs, and exports.
- Coverage and test health signals suitable for local developer feedback and CI.

The module also produces indirect outputs: confidence in behavior changes, protection against accidental regressions, and a stable mechanism for updating expected artifacts when intentional changes occur.

9. Internal Design

Main subcomponents

Subcomponent	Purpose	Key constraint
Unit test layer	Verifies functions and classes in isolation	Must avoid deep fixture dependency
Integration test layer	Verifies module-to-module contracts	Must use canonical trace objects, not raw framework objects
Fixture library	Stores representative inputs and expected outputs	Must remain small, curated, and reviewable
Golden comparison utility	Compares structured outputs and rendered artifacts	Must normalize irrelevant formatting differences
Test runner conventions	Defines how tests are invoked locally and in CI	Must be deterministic and document the subset strategy
Regression suite	Protects known bug fixes and edge cases	Must include the minimal reproducible case, not a giant dump

Workflow

Source change
-> Targeted unit tests
-> Module integration tests
-> Golden artifact comparison
-> Regression suite
-> CI gate

-> Merge only if deterministic and green

The workflow is intentionally layered. Fast unit checks should fail first, integration tests should prove that boundaries still hold, and golden/regression tests should confirm that observable behavior did not drift.

Important internal decisions

- Use deterministic fixtures as the source of truth for behavior checks.
- Separate semantic assertions from presentation assertions. A report can be semantically correct but visually wrong, and both matter.
- Prefer small, explicit tests over broad end-to-end tests that obscure the failing boundary.
- Keep comparison helpers strict about important fields and tolerant only where formatting differences are expected.

10. Data Model

The testing layer owns a small set of support entities. These are not product entities; they are verification artifacts.

Entity	Key fields	Why it exists
Fixture	id, type, source, version, canonical reference	Provides repeatable input for a test
Scenario	name, module, behavior, expected outcome	Explains what the test proves
Golden artifact	artifact type, content hash, normalization rules, baseline version	Captures stable expected output
Comparison rule	allowed drift, strict fields, ignored fields	Defines what matters in an assertion
Regression case	bug id, minimal reproduction, fix version	Prevents a known failure from returning

Relationships are simple: scenarios reference fixtures; fixtures point to expected artifacts; comparison rules define how the artifact is checked. This model exists to keep tests readable and intentional instead of becoming piles of anonymous files.

11. Algorithms / Processing Logic

Step-by-step processing

1. Select the smallest test scope that exercises the changed boundary.
2. Load the fixture and normalize any environment-sensitive fields.
3. Run the module under test.
4. Compare actual output against the expected artifact using the comparison rule for that artifact type.
5. If the output is structural, compare semantically; if it is rendered, compare visually or via normalized text as appropriate.

6. Report a failure with the smallest reproducible context that explains the mismatch.

Edge cases

- Timestamp and path fields must be normalized before comparison.
- Ordering must be deterministic for collections whose order is not semantically meaningful.
- Floating-point and formatting differences should be constrained to the smallest safe tolerance.
- Large binary or rendered artifacts should be compared through defined baselines rather than ad hoc string diffs.

Decision rules

- Fail fast when the fixture itself is invalid.
- Fail on semantic drift even if the output still looks plausible.
- Allow formatting drift only where the comparison policy explicitly permits it.
- Update goldens only through deliberate review, never implicitly during a passing test run.

Deterministic behavior is a requirement. If a test outcome depends on ambient state, the test is wrong, not the developer machine.

12. Tradeoffs

Tradeoff	Chosen approach	Why
Simplicity vs flexibility	Curated fixtures and explicit comparisons	Easier to maintain than a generic test DSL
Generality vs specificity	Project-specific testing utilities	The repository is one product, not a testing platform
Determinism vs heuristic inference	Strict deterministic comparisons	Engineers need reliable, explainable failures
Snapshot vs hand-written assertions	Use both, but only where each is strongest	Structured outputs benefit from snapshots; critical semantics need explicit assertions

The project should resist the temptation to over-engineer a universal test framework. The right answer is a narrow, opinionated, boring set of tools that make the real code easy to trust.

13. Risks and Failure Modes

- Fixture drift: expected outputs become stale after intentional changes.
- False confidence: broad tests that pass while missing important semantic regressions.
- Flaky comparisons: tests that tolerate too much or depend on environment-specific state.
- Golden bloat: too many large files make review and maintenance painful.
- Hidden coupling: shared helpers become a second implementation of business logic.
- Regression blindness: bug fixes are not captured as minimal reproducible tests.

The main hidden complexity is not the mechanics of running tests. It is keeping tests small, honest, and aligned with the actual risk surface of each module.

14. Testing Strategy

Unit tests

- Verify pure functions, boundary checks, normalization helpers, and comparison utilities.
- Use the smallest fixture or inline value needed to express the rule.
- Keep unit tests fast enough to run continuously during development.

Integration tests

- Verify contracts between parser, normalizer, canonical trace model, analysis engine, postmortem generator, graph generation, and render/export layers.
- Use canonical trace fixtures and golden outputs to prove boundary compatibility.
- Cover file I/O and local orchestration without invoking external services.

Fixture-driven tests

- Use curated corpus files that represent real failure patterns, edge cases, and minimal reproductions.
- Store the fixture plus the expected outputs together so the intent is obvious.
- Prefer a few excellent fixtures over a large archive of random examples.

Regression protection

- Every fixed bug should gain a regression test with a minimal reproducer.
- Goldens should be updated through a deliberate review process.
- CI must block merges when structural outputs or critical render artifacts drift unexpectedly.

The strategy is intentionally layered: fast tests first, then contract tests, then golden regressions. That sequence minimizes feedback time while still protecting the observable behavior that users care about.

15. Implementation Checklist

7. Create a central test directory structure with explicit subdirectories for unit, integration, fixtures, and goldens.
8. Define naming conventions for fixtures, scenarios, and regression cases.
9. Implement comparison helpers for structured JSON, markdown, graph artifacts, and rendered files.
10. Add normalization rules for timestamps, path separators, line endings, and collection ordering.
11. Create a minimal set of canonical fixtures that represent the main pipeline and key failure modes.
12. Introduce a golden update workflow that requires review and prevents accidental overwrites.
13. Document local commands for running the whole suite and targeted subsets.
14. Wire the test suite into CI with fast-fail stages and clear reporting.
15. Add regression tests for every bug fixed during MVP development.
16. Audit helpers periodically to ensure they are not duplicating product logic.

If this checklist is followed, the repository will have a stable test backbone instead of a pile of one-off assertions attached to whichever feature happened to be built last.

16. Future Expansion

- Property-based testing for parser and normalization edge cases.
- Fuzzing for malformed trace inputs and maliciously shaped JSON.
- Performance benchmarks for large traces and large report artifacts.
- Browser-level UI automation for the web interface.
- Multi-run comparison fixtures for drift analysis.
- Broader corpus management with labeled real-world trace samples.

These are sensible follow-ons, but they should be introduced only after the MVP suite is stable and the team has observed where the real maintenance cost lies.

17. Final Recommendation

This design is MVP-ready. It is intentionally narrow, deterministic, and aligned with the rest of the architecture. It treats testing as a product-quality system rather than an afterthought.

The main simplification still recommended is to keep the initial fixture corpus small. The best test infrastructure in the world will still become annoying if it starts with too many goldens and too much ceremony.